DevSys VM — OrbStack Golden-Clone Runbook

Identical, version-pinned Ubuntu dev VMs on a Mac with OrbStack — build one golden base machine, clone it per developer in seconds, reach each clone over Tailscale.

The system: three scripts in the `devsys-orbstack` folder. `build-base.sh` and `newvm.sh` run on the Mac; `devsys-vm.sh` (installer 3.3.0) runs inside the guest. You install the pinned toolchain once into a base machine called `devsys-base`, and every developer VM is a clone of it. Each clone joins Tailscale tagged `tag:devsys` + a tag of its own; the admin reaches every VM, each developer reaches only their own.

Each VM runs **Ubuntu 26.04 LTS (resolute)**, arm64 on Apple Silicon, on OrbStack's shared Linux kernel.

Phase	What	How often	Where
0	Prepare the Mac (OrbStack, the scripts)	once	the Mac
1	Tailscale tags, SSH rules, OAuth client	once, plus one tag and rule per VM	Tailscale admin console
2	Build the golden base	once, and again to change tool versions	the Mac
3	Clone a developer VM	per developer	the Mac
4	Verify and operate	ongoing	the Mac / inside a VM

Phase 0 — Prepare the Mac (one-time)

0.1 Install OrbStack

Install OrbStack (`brew install orbstack`, or the download from orbstack.dev) and open it once so that `orbctl` is on the PATH. Check both tools the scripts need:

```
orbctl version
jq --version
```

The VMs run only while OrbStack is running. Set OrbStack to start at login and keep the Mac from sleeping, or developers lose their VMs when it does.

0.2 Get the scripts

```
git clone https://github.com/sohebcakewalk/dev-setup.git
cd dev-setup/devsys-vm/devsys-orbstack
```

0.3 Tailscale on the Mac (optional)

Install the Tailscale app and sign in as a tailnet admin if you want to `ssh` into the VMs from this Mac. Without it you can still get a root shell in any VM with `orbctl run`.

Phase 1 — Tailscale setup (one-time, tailnet-level)

All in the Tailscale admin console. The tags and rules are the same ones the Incus setup uses.

1.1 Tags (Definitions → Tags)

Tag	Owner
`tag:devsys`	`autogroup:admin, tag:devsys`
`tag:beehivekk`	`tag:devsys`
(one per further VM)	`tag:devsys`

`tag:devsys` must own itself — a key carrying two tags needs every tag, including `tag:devsys`, to pass the ownership check. Without it `newvm.sh` stops with “Auth key mint failed”.

1.2 Tailscale SSH rules (Access controls → Tailscale SSH)

#	Sources	Destination	Users
1	`autogroup:member`	`autogroup:self`	`autogroup:nonroot`, `root`
2	`autogroup:admin`	`tag:devsys`	`root`
3	`bh@github`	`tag:beehivekk`	`autogroup:nonroot`
+1 per VM	`<login>@github`	`tag:<vm>`	`autogroup:nonroot`

Rule 2 is the admin's root reach across every VM; rules 3 and up let each developer into only their own VM. Never use `tag:devsys` as a source — it would let every VM reach every other VM.

Sources is the developer's Tailscale login as the Users page shows it, for example `name@github` for someone who signed up with GitHub. `autogroup:nonroot` lets them log in as any Linux user on their VM except `root`, so usernames can be chosen freely; list names instead (`bh`, `admin`) to restrict it.

1.3 OAuth client, stored on the Mac

The OAuth client is what lets `newvm.sh` create a fresh Tailscale auth key for each VM, so nobody has to generate and paste keys by hand. Create it once: Settings → Trust credentials → Credential → OAuth, scope Auth Keys: Write, tag `tag:devsys`.

That one tag is enough for every developer. A client may mint a key for any tag its own tag owns, and `tag:devsys` owns itself and each VM tag (1.1). This follows Tailscale's documentation and has not been run here; if minting is refused for a new developer, add their tag to the client as well.

Store the client ID and secret in the macOS Keychain:

```
security add-generic-password -s ts-oauth -a tailscaleclientid -w '<id>'
security add-generic-password -s ts-oauth -a tailscaleclientsecret -w '<secret>'
```

`newvm.sh` looks for the client in this order: the environment (`DEVSYS_TS_CLIENT_ID`, `DEVSYS_TS_CLIENT_SECRET`), then the mode-600 file `~/.config/devsys/ts-oauth.env` defining the same two variables, then the Keychain items above.

The secret is long-lived: anyone who can read it can mint keys for these tags. Keep it only in the Keychain or the mode-600 file, and revoke the client in Tailscale if the Mac is ever compromised.

Phase 2 — Build the golden base (one-time)

```
./build-base.sh
```

One command does the whole phase:

1. Creates the machine `devsys-base` from `ubuntu:resolute` (Ubuntu 26.04 LTS), with `--isolated --isolate-network` so the Mac's home folder and the other machines' files are hidden. Clones inherit this.

2. Copies the `devsys-vm.sh` beside it into the machine and installs every group, including the inbound firewall — no developer user, no Tailscale login.
3. Clears the per-machine identity (machine-id, Tailscale state, SSH host keys).
4. Stops the machine.

On an M1 Pro this took about 3 minutes and 3.4 GB. Check the result with `orbctl list`: `devsys-base` must show as stopped.

The stopped base is the image. Do not start it by hand: a running base regenerates the identity that clones must not share, and `newvm.sh` refuses to clone it. To change it, re-run `./build-base.sh`.

Option (environment variable)	Effect
`DEVSYS_GROUPS="node python ai"`	install only these groups into the base
`DEVSYS_SHARED_CLAUDE=1`	bake in the shared-Claude setup (several Unix users, one Claude login)
`DEVSYS_BASE_IMAGE=ubuntu:noble`	use another image for a new base, here Ubuntu 24.04 LTS (default: `ubuntu:resolute`)
`./build-base.sh devsys-base-next`	build under a different base name

Phase 3 — Clone a developer VM (per developer)

3.1 Checklist for each developer

1. Tailscale → Tags: add `tag:<vm>`, owned by `tag:devsys` (1.1).
2. Tailscale → SSH rules: add the developer's Tailscale login → `tag:<vm>` → `autogroup:nonroot` (1.2).
3. On the Mac, in the `devsys-orbstack` folder:

```
./newvm.sh beehivekk bh --tag beehivekk
```

The first argument is the VM name (lowercase letters, digits, hyphens). The second is the first Linux username to create; it can be anything (`bh`, `admin`, `nv`). `--tag` names the Tailscale tag, here after the VM, so the SSH rule points at this one VM whatever its usernames are. Without `--tag` the tag is named after the username (`tag:bh`), which only works while every VM has a different username.

Repeat the three steps for every developer; nothing on the Mac or in the OAuth client changes between them. A developer who should own their VM under their own Tailscale account can join with a login link instead (3.4).

3.2 What `newvm.sh` does

1. **Checks first.** The base must exist and be stopped, and no machine may already have that name. A failed check stops the script before anything is asked of Tailscale.
2. **Reads the OAuth client** from the environment, the mode-600 file, or the Keychain (1.3).
3. **Gets a temporary access token** from Tailscale with that client.
4. **Mints an auth key** tagged `tag:devsys` + `tag:beehivekk`: single-use, not ephemeral, pre-approved, valid for one hour.
5. **Clones** `devsys-base` and starts the clone.
6. **Sets up the clone**: clears any Tailscale state, creates the user, and joins the tailnet with `tailscale up --ssh --accept-dns=false --accept-routes=true`.

The key never appears on screen, in a file, or in a command line; it is handed to the clone on stdin. The second tag is the one given with `--tag`, or the username when `--tag` is left out.

Key setting	Why it matters
Not ephemeral	an ephemeral VM is removed from the tailnet whenever it goes offline
Both tags	`tag:devsys` gives the admin root reach; `tag:<vm>` is what the developer's SSH rule points at
Pre-approved	the VM joins without waiting for device approval
Single-use, one hour	a leaked key cannot add a second machine

`--accept-dns=false` keeps the VM on its own DNS, so Tailscale's MagicDNS cannot break ordinary lookups such as npm. You still reach the VM by name from your Mac. To let one VM look up other tailnet machines by short name, run `tailscale set --accept-dns=true` inside it.

To clone from a different base machine, add `--base NAME`. If setup inside the clone fails, the VM is left in place: inspect it with `orbctl run -m <vm> -u root bash`, or remove it with `orbctl delete <vm>` and run `newvm.sh` again.

3.3 A second user on the same VM

To add another Linux user to an existing VM, for example `admin` beside `bh`:

```
orbctl run -m beehivekk -u root \
  env DEVSYS_UNATTENDED=1 DEVSYS_USER=admin devsys install none
```

The new user gets the same shell setup, `~/work` folder and tools. Tailscale is not touched. A rule whose users are `autogroup:nonroot` (1.2) already lets the developer log in as the new user. If your rule lists usernames instead, add this one to it.

Every user the script creates has passwordless `sudo`, so `admin` has no more rights than `bh`. To make one of them an ordinary user, remove its sudo access:

```
orbctl run -m beehivekk -u root \
  bash -c 'rm -f /etc/sudoers.d/90-devsys-bh; gpasswd -d bh sudo'
```

That user can then no longer run the `devsys` subcommands that need root (`mise`, `npm`, `serve`, `fix-dns`). Claude, GitHub and git logins are separate for each user.

3.4 Alternative: the developer joins with a login link

Use this when a developer should own their VM under their own Tailscale account. It needs no OAuth client, no tag and no per-developer SSH rule; the developer only has to be a member of the tailnet already (Users → Invite).

```
./newvm.sh beehivekk bh --login-link
```

The script clones the VM and creates the user as before, but mints no key. It prints a `login.tailscale.com/a/…` link. Send it to the developer; when they open it and sign in, the VM joins the tailnet as their device. If no link is printed, fetch one with `orbctl run -m beehivekk -u root devsys ts-login`.

	Automatic key (3.1)	Login link (3.4)
VM owner in Tailscale	the tailnet (tagged `tag:devsys` + `tag:<vm>`)	the developer
Tailscale setup per developer	one tag and one SSH rule	none, just the invite
Developer's SSH access	their own rule (rule 3)	rule 1: members reach their own devices
Admin `ssh root@<vm>`	yes (rule 2)	no: the VM has no `tag:devsys`; use `orbctl run` from the Mac
Who finishes the join	the script	the developer, in a browser
Key expiry	off for tagged devices	the tailnet's normal expiry applies; the developer re-authenticates when it lapses

The two can be mixed: some VMs tagged, some developer-owned. Creating the VM with this flag was tested on OrbStack; the browser sign-in that completes the join was not recorded as tested.

Phase 4 — Verify and operate

Check / task	Command
List machines and their state	`orbctl list`
Admin reach (rule 2)	`ssh root@beehivekk`
Dev login (rule 3)	`ssh bh@beehivekk`
Isolation (must fail)	`ssh bh@`<another dev's VM> → denied
Firewall is on	`orbctl run -m beehivekk -u root systemctl is-active devsys-firewall` → `active`
Health and versions	`orbctl run -m beehivekk -u root devsys status`
Break-glass root shell	`orbctl run -m beehivekk -u root bash`
Change tool versions	edit the `MISE_*`/`NPM_*` pins in `devsys-vm.sh` → `./build-base.sh` → new clones get them; existing clones are not touched
Retire a VM	`orbctl delete beehivekk`, then remove it from Tailscale → Machines

Do not run `devsys update` inside a VM yet. It downloads the published gist copy, which is still 3.1.6, and replaces 3.3.0 with it. That is safe again once `devsys-vm.sh` is republished to the gist as `devsys-v3.sh`.

Ollama on the Mac, used from the VMs (optional)

A model served by Ollama on the Mac can be shared with every VM. The VMs only call Ollama's API over the network; they still cannot see the Mac's files.

On the Mac (once)

1. In the macOS account that runs Ollama, make it listen on the network, then quit and reopen Ollama:

   ```
   launchctl setenv OLLAMA_HOST "0.0.0.0:11434"
   ```

2. Find the Mac's LAN address with `ipconfig getifaddr en0`, and reserve it in the router so it does not change. Use this address, not a Tailscale name: an isolated VM cannot reach the Mac by its OrbStack host name, and the Mac may be on a different tailnet from the VMs.

The setting is lost on reboot, and Ollama runs only while that macOS account is logged in. After a restart, log in to it, run the command again, then open Ollama.

For each VM

```
./newvm.sh beehivekk bh --tag beehivekk --ollama 192.168.1.50
```

`--ollama` sets `OLLAMA_HOST=http://192.168.1.50:11434` for every user of that VM, in zsh and bash, including users added later. The golden base is not changed, so there is nothing to rebuild. To avoid typing it each time, `export DEVSYS_OLLAMA_HOST=192.168.1.50` on the Mac before running `newvm.sh`. For a VM that already exists:

```
orbctl run -m beehivekk -u root bash -c '
  f=/etc/profile.d/devsys-ollama.sh
  echo "export OLLAMA_HOST=http://192.168.1.50:11434" > $f
  for rc in /etc/zsh/zshenv /etc/bash.bashrc; do
    echo "[ -f $f ] && . $f" >> $rc
  done'
```

Using it inside a VM

Check the connection, which also lists the models the Mac offers:

```
curl $OLLAMA_HOST/api/tags
```

Claude Code can run against it, because Ollama 0.14 and later speaks Anthropic's API:

```
ANTHROPIC_BASE_URL=$OLLAMA_HOST ANTHROPIC_AUTH_TOKEN=ollama ANTHROPIC_API_KEY="" \
  claude --model <model-name>
```

Use a model name exactly as `/api/tags` lists it. Ollama's documentation asks for a model that supports tool calling and a context window of 64k tokens or more; with less, Claude Code cannot edit files or loses track of the task. Tools that speak the OpenAI format use `$OLLAMA_HOST/v1` as their base URL.

Ollama has no password. Listening on `0.0.0.0` opens it to every VM and every device on the Mac's network. All developers share one model on one machine, so heavy use by one slows the others. The `--ollama` option was tested with a build and clones; Claude Code against a remote Ollama follows Ollama's documentation, which only shows the same-machine case, and was not run here.

What a developer can and cannot reach

Checked on OrbStack 2.2.4 with a base and two clones, as root inside a clone.

From inside a VM	Result	What enforces it
The Mac's files (`/mnt/mac`, `/Users`, `/Volumes`)	not present	`--isolated`
Other VMs' files (`/mnt/machines`)	not present	`--isolated`
Mac commands, the Mac's SSH agent, the Docker socket	not available	`--isolated`
A port on another VM	blocked	the inbound firewall in each VM
Services on the Mac, via the Mac's LAN address	**reachable**	nothing (see below)
The internet	reachable	intended

`--isolate-network` alone does not separate the VMs: with it set, one clone still connected to a port on another. So installer 3.3.0 gives every VM its own firewall. It drops new incoming connections except over Tailscale and loopback, leaves outgoing traffic alone, is baked into the base, starts at boot, and is reported by `devsys status`. The rules are in `/etc/devsys/firewall.nft`.

A dev server is therefore reachable over Tailscale (`devsys serve <port>`, or the VM's tailnet address) but not from other VMs, and not from the Mac at `<vm>.orb.local`. `DEVSYS_FIREWALL=0 ./build-base.sh` builds a base without the firewall.

What is left open:

- **The Mac's own network services.** A VM can connect to anything the Mac offers on its LAN address (Remote Login, Screen Sharing, File Sharing). Turn off the ones you do not need in System Settings → General → Sharing.
- **Root inside the VM.** Every developer has passwordless `sudo` and can stop their own firewall. That exposes only their own VM. Two users on one VM can read each other's files.
- **The shared kernel.** OrbStack says isolated machines are not a full security boundary. Use the Incus setup when developers must be strongly separated.
- **Tailscale's network policy.** The SSH rules decide who can log in; whether one VM can reach another's ports over the tailnet is decided by the tailnet's access rules, which allow everything by default.

When something stops

Message	Fix
`orbctl not found`	install OrbStack and open it once (Phase 0.1)
`No base machine 'devsys-base'`	run `./build-base.sh`
`Base 'devsys-base' is running, not stopped`	re-run `./build-base.sh`; do not stop it by hand, its identity has to be cleared again
`A machine named '…' already exists`	pick another name, or `orbctl delete` the old one
`No Tailscale OAuth client`	store the client ID and secret (Phase 1.3)
`OAuth token exchange failed`	the stored ID or secret is wrong or revoked
`Auth key mint failed`	the VM tag is missing or not owned by `tag:devsys`, or `tag:devsys` does not own itself (Phase 1.1). If both are right, add the VM tag to the OAuth client (1.3)
“not permit you to SSH as user X”	no SSH rule lists that username for this VM's tag (Phase 1.2, 3.3)

Moving a developer over from the Incus server

An Incus VM cannot be imported into OrbStack. Build the VM fresh with `newvm.sh` under a different Tailscale name, then copy the home folder through an admin machine that reaches both VMs as root:

```
EXCLUDES='--exclude=node_modules --exclude=.venv --exclude=.cache'
ssh root@<old-vm> "tar -C /home/bh $EXCLUDES -czf - ." \
  | ssh root@<new-vm> 'tar -C /home/bh -xzpf -'
```

Reinstall `node_modules` and `.venv` in each repo afterwards. Do not copy `/opt/devsys/mise`, `/var/lib/tailscale` or `/etc/machine-id`. Once the old VM is retired, rename the new one inside the VM with `tailscale set --hostname=<name>`. This recipe has not been run against the Incus server.

Tested on an Apple Silicon Mac (M1 Pro, OrbStack 2.2.4): `build-base.sh` built a base with all 12 groups; `newvm.sh`, run without a Tailscale key, cloned it in 6 seconds with its own hostname and machine-id, the dev user, no `/mnt/mac`, and every tool at its pinned version. With installer 3.3.0, a connection from one clone to a port on the other was refused in both directions, went through when one firewall was stopped, and was refused again after a restart. The same build and clone checks passed on Ubuntu 26.04 LTS, the default image.

Not yet tested: the Mac Studio itself; steps 2 to 4 and the Tailscale join in 3.2 (minting a real key and joining the tailnet — that code is carried over from the Incus `newvm.sh`); the firewall on a VM that has joined a tailnet (Tailscale SSH and `devsys serve` through it); the second-user commands in 3.3 on OrbStack; `DEVSYS_SHARED_CLAUDE=1` on OrbStack; the migration recipe. Try the first VM end to end before creating the rest.